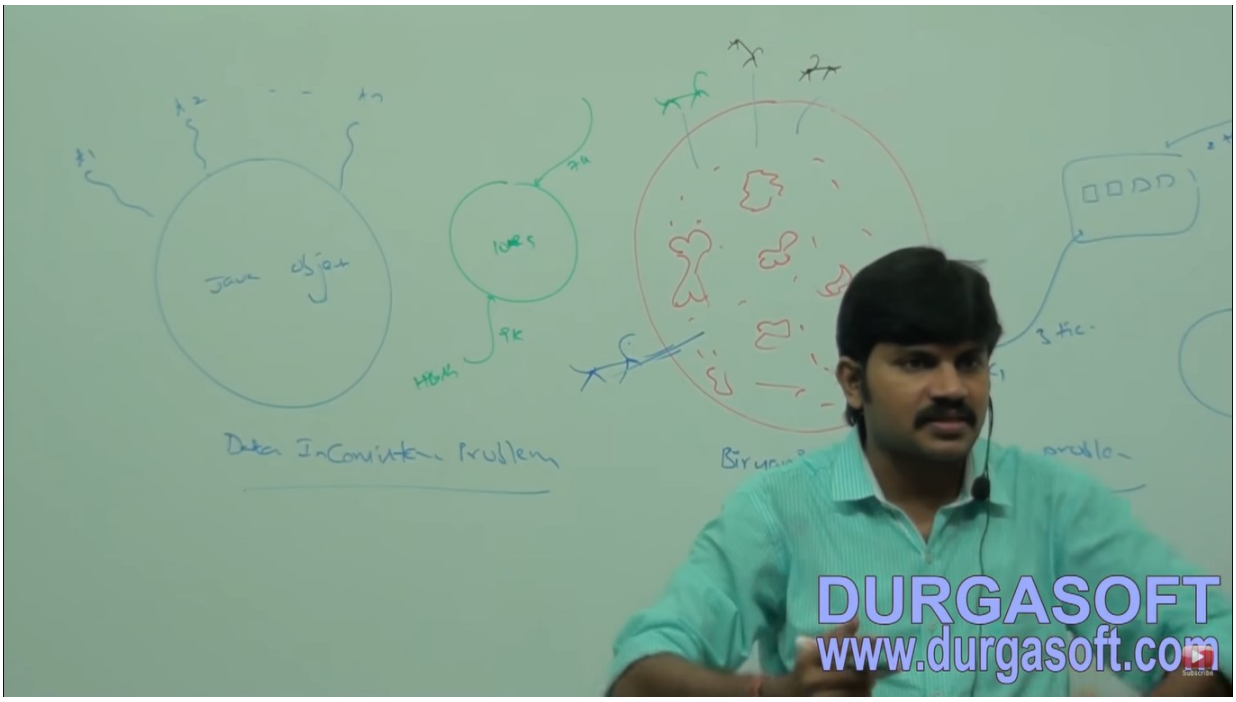
SYNCHRONIZATION

(1) SYNCHRONIATION STORY :



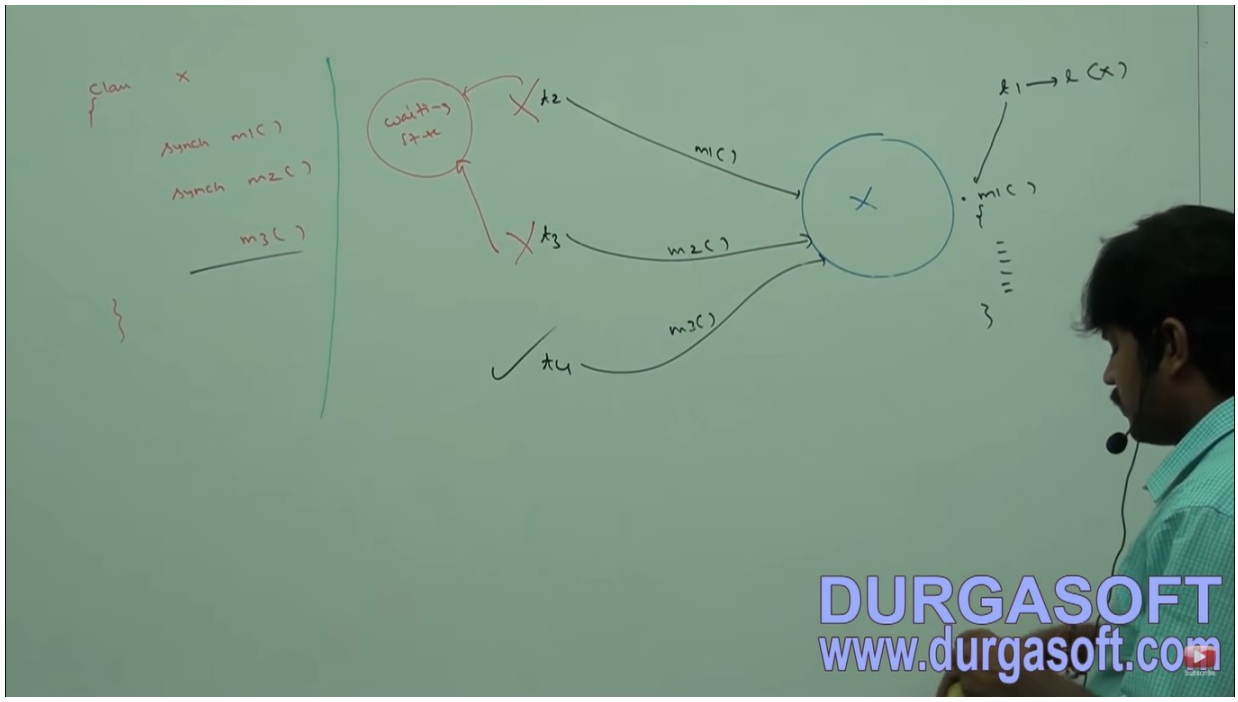
- Synchronizer is a modifier applicable only for methods and blocks but not for classes and variables.
- If multiple threads are trying to operates simultaneously on the same java object then there may be chance of data inconsistency problem to over come this problem we should go for synchronization keyword
- If a method or block declare as synchronizer then at a time only one thread is allowed to execute that method or block on the given object so that data inconsistency problem will be resolved.

- The main advantage of synchronizer keyword is we can resolve data inconsistency problems but main disadvantage of synchronizer keyword is it increases waiting time of threads and creates performance problems. Hence if there is no specific requirement then it is not recommended to use synchronizer keyword.

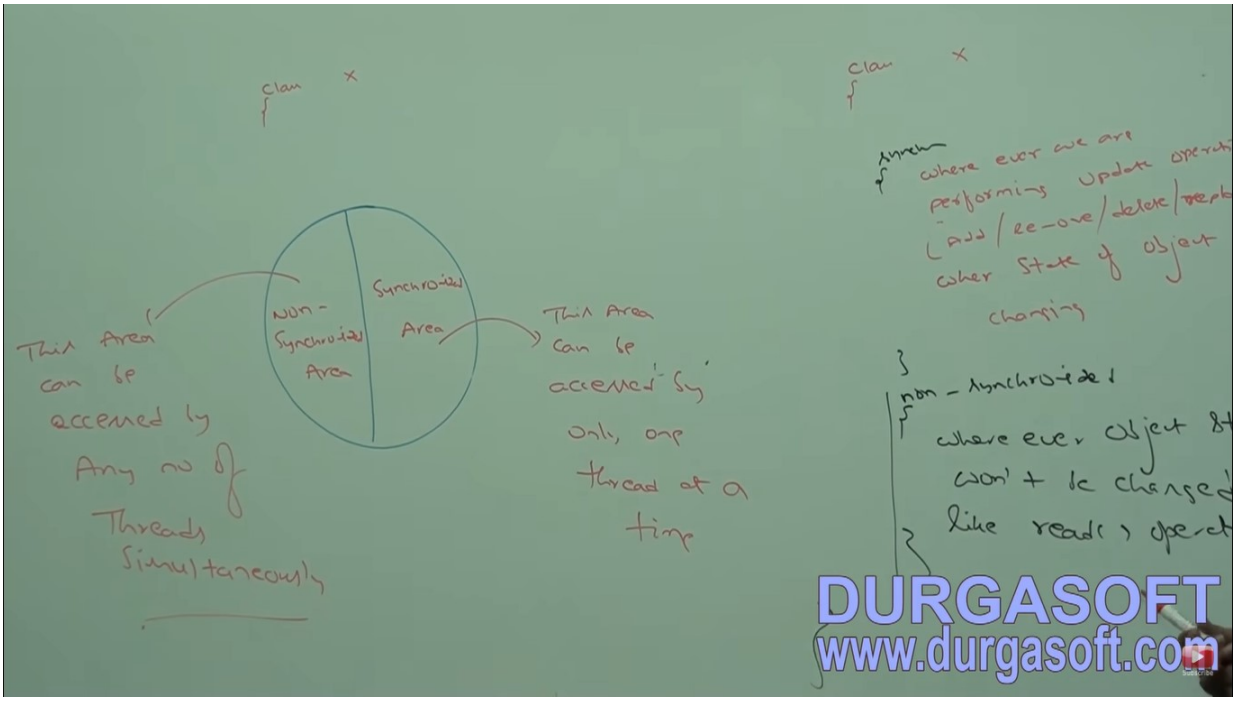
- Internally synchronization concept is implement by using lock every object in java has a unique.

- Whever we are using synchronizer keyword then only lock concept will come into the picture.

- If a thread want to execute synchronizer method on the given object first it has to get lock of that object .
- Once thread got the lock then it is allowed to execute any synchronizer method on that object.
- Once methods execution completes automatically thread release the lock
- Acquiring and releasing lock internally takes care by JVM and programmer not responsible for this activity .
- While a thread executing synchronizer method on the given object the remaining thread are not allowed to execute any synchronizer method simultaneously on the same object but remaining threads are allowed to execute non-sysnchronizer method simultaneously

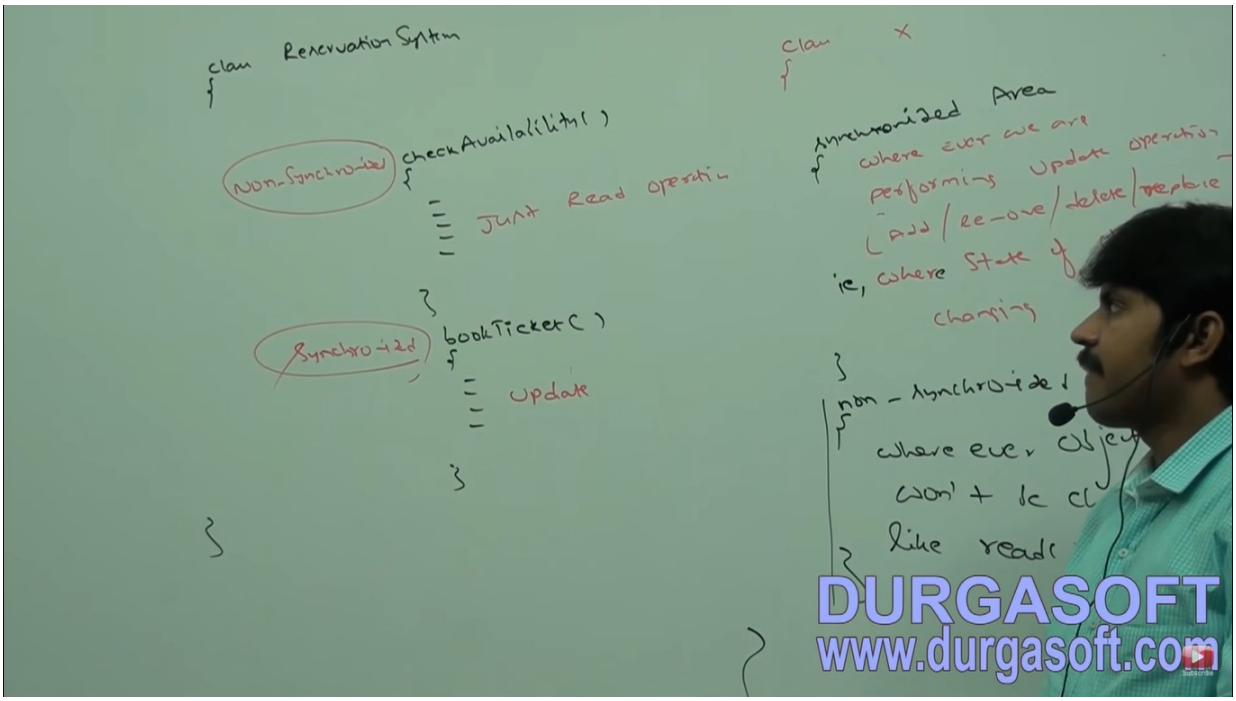


Lock concept is implemented based on object , but not based on methods.

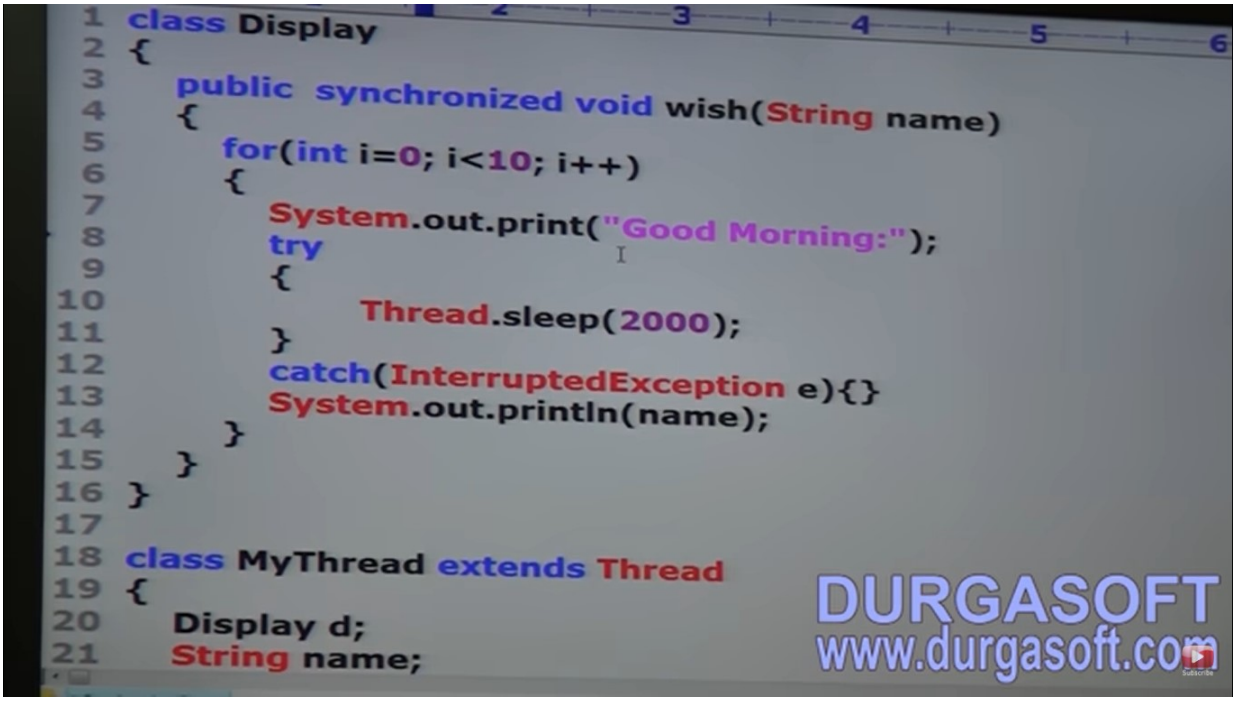


Ex-

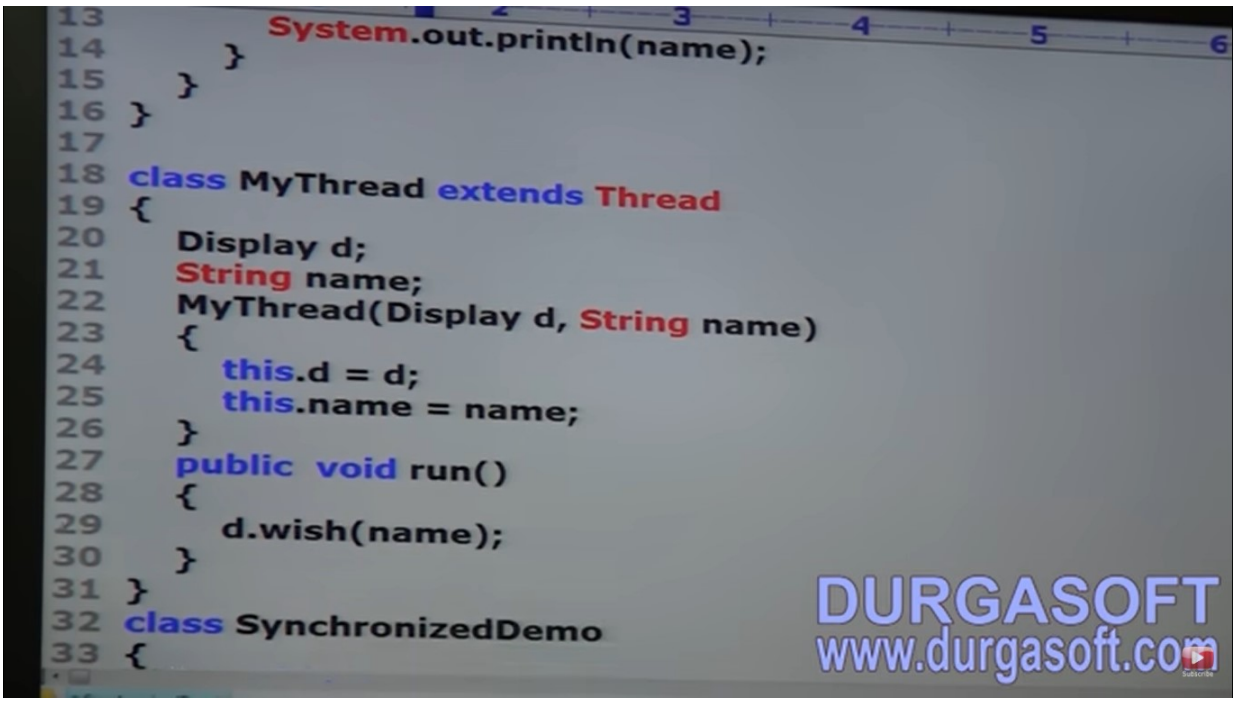
When go for synchronization & when not



Example with synchronization 1:



```
class Display
{
   public synchronized void wish(String name)
   {
      for(int i=0; i<10; i++)
      {
         System.out.print("Good Morning:");
         try
         {
            Thread.sleep(2000);
         }
         catch(InterruptedException e){}
         System.out.println(name);
      }
   }
}

class MyThread extends Thread
{
   Display d;
   String name;
```

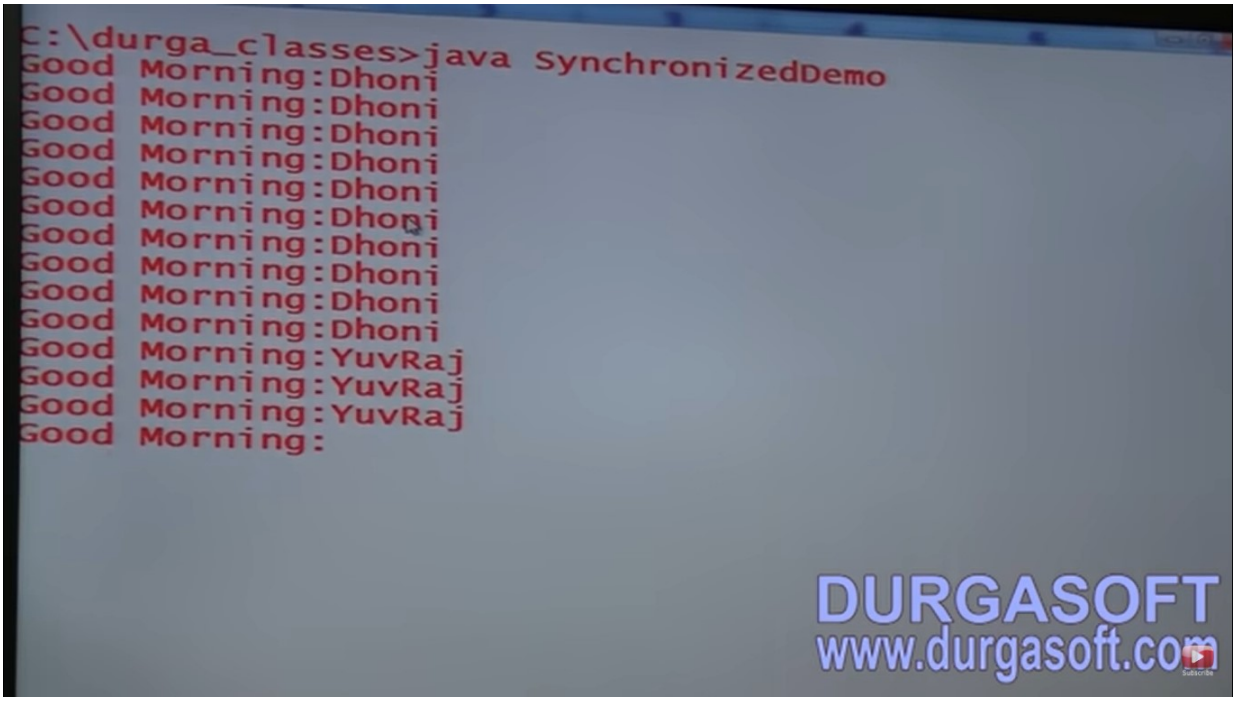
```
         System.out.println(name);
      }
   }
}

class MyThread extends Thread
{
   Display d;
   String name;
   MyThread(Display d, String name)
   {
      this.d = d;
      this.name = name;
   }
   public void run()
   {
      d.wish(name);
   }
}
class SynchronizedDemo
{
```

```
        this.d = d;
        this.name = name;
    }
    public  void run()
    {
        d.wish(name);
    }
}
class SynchronizedDemo
{
    public static  void main(String[] args)
    {
          Display d = new Display();
          MyThread t1 = new MyThread(d,"Dhoni");
          MyThread t2 = new MyThread(d,"YuvRaj");
          t1.start();
          t2.start();

    }
}
```

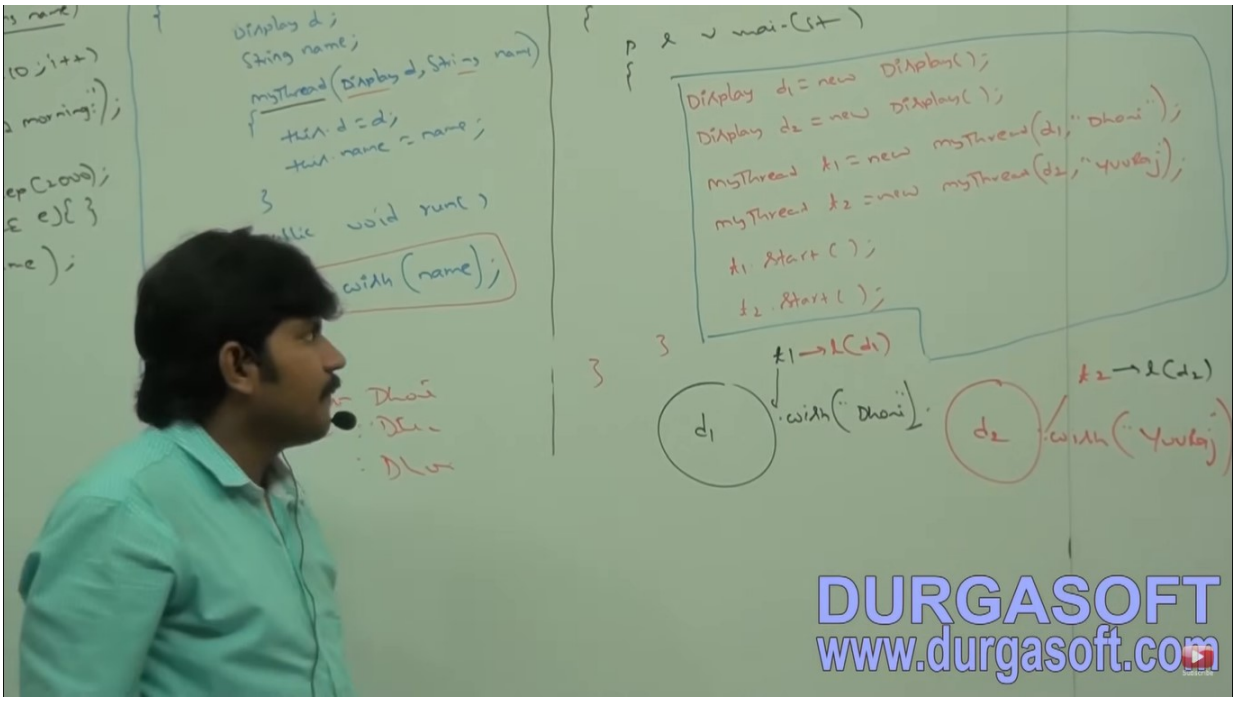

If we not declare wish method as synchronized then both thread will be executed simultaneously and hence we will get irregular output

```
:\durga_classes>javac SynchronizedDemo.java

:\durga_classes>java SynchronizedDemo
ood Morning:Good Morning:YuvRaj
ood Morning:Dhoni
ood Morning:YuvRaj
ood Morning:Dhoni
ood Morning:
:\durga_classes>_
```


If we declare wish method as synchroniezed then at a time only one thread is allowed to execute wish method on the given display object hence , we will get regular output .



Case study:



Even though wish method is synchronized we will get irregular output because threads are operating on different java objects.

Reason:

If multiple threads are operating on same java objects then synchronization is required if multiple threads are operating on multiple java objects then synchronization is not required.

(2) CLASS LEVEL LOCK:

- Every class in java has a unique lock which is nothing but class level lock .
- If a thread wants to execute a static synchronization method then thread required class level lock.
- Once a thread got class level lock then it is allowed to execute any static synchronnation method of that class.
- Once a method execution complete automatically threads releases the lock.

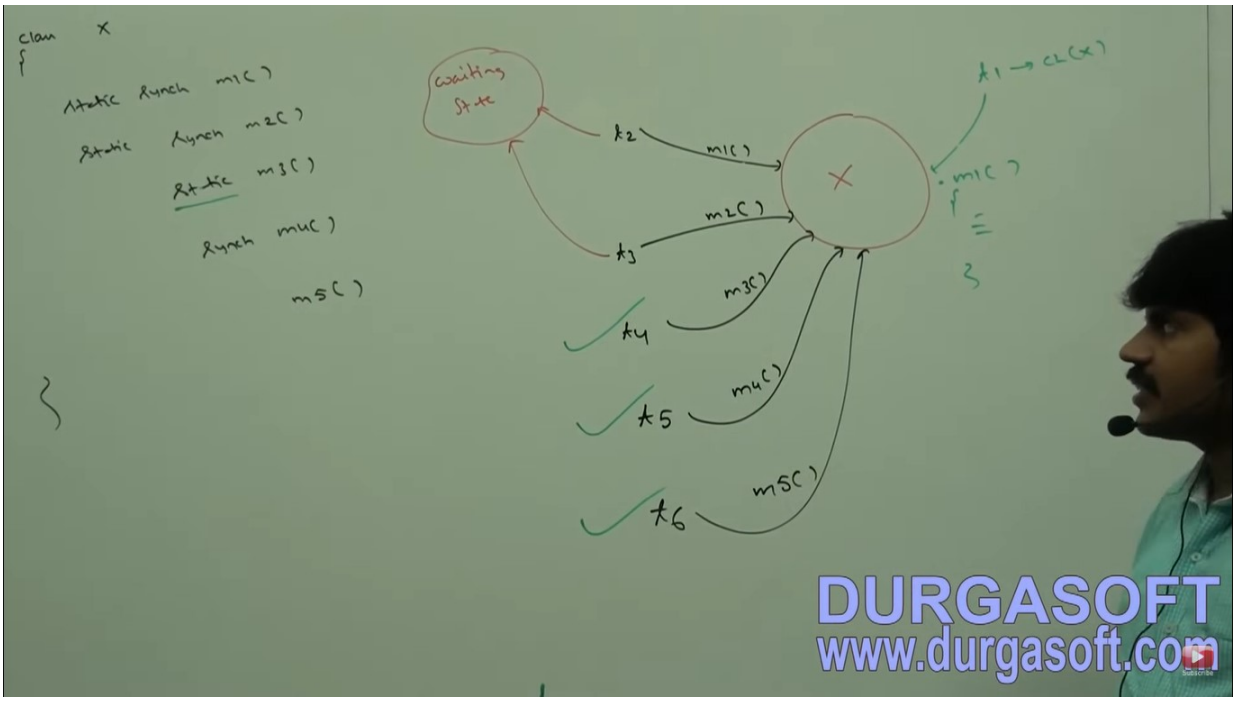
- While a thread executing static synchronization method the remaining threads are not allowed to execute any static synchronier method of that class simultaneuously but remaing threads are allowed to execute the following methods simultaneously,

  (a) normal static methods

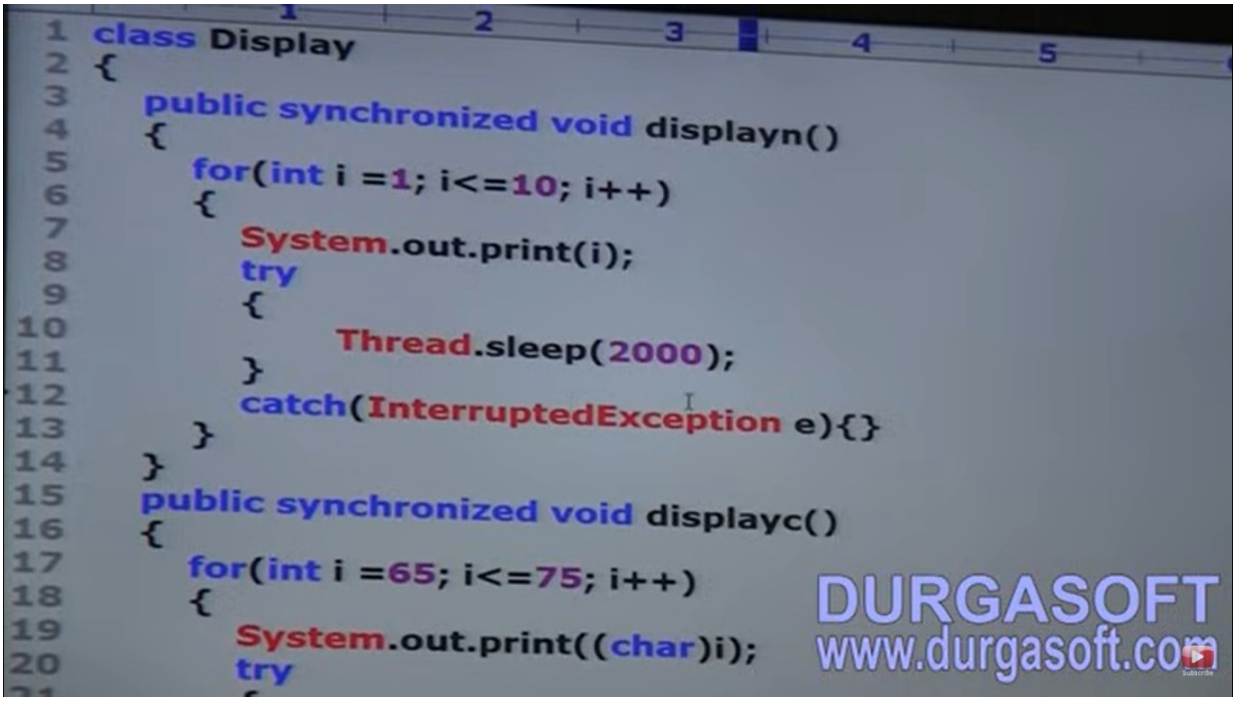
  (b) synchroned instance methods

  (c) normal instance methods

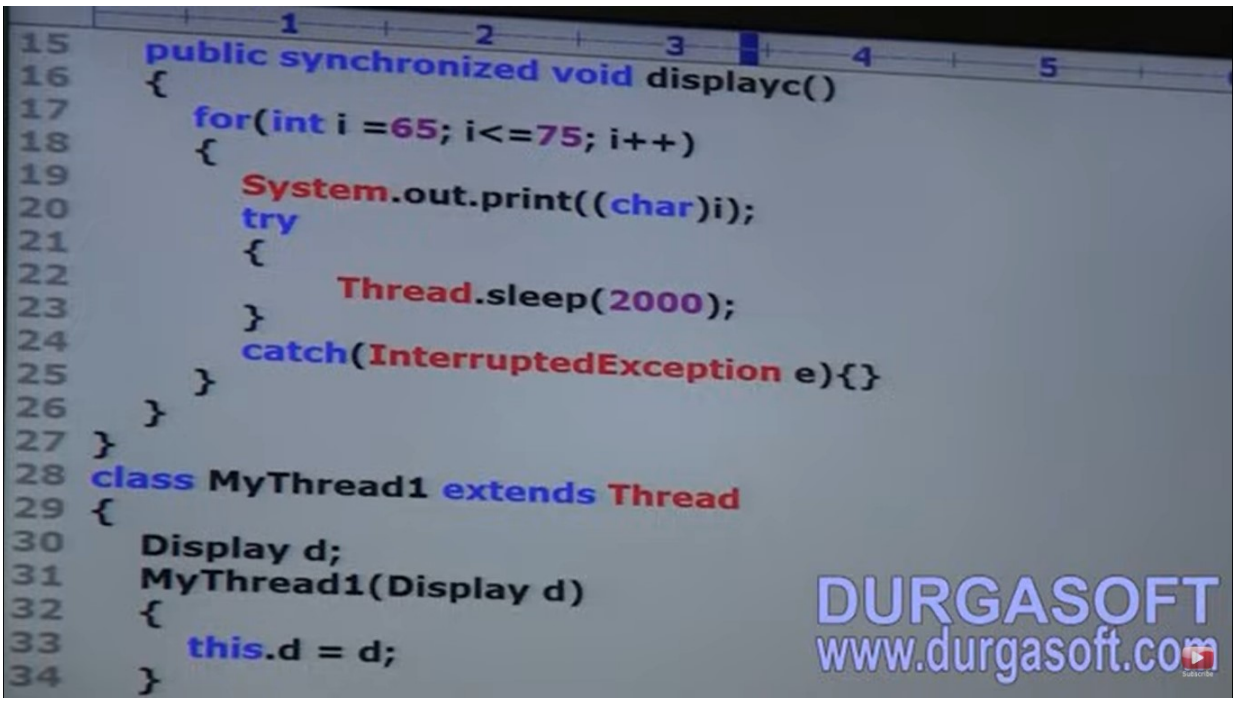
  Ex-



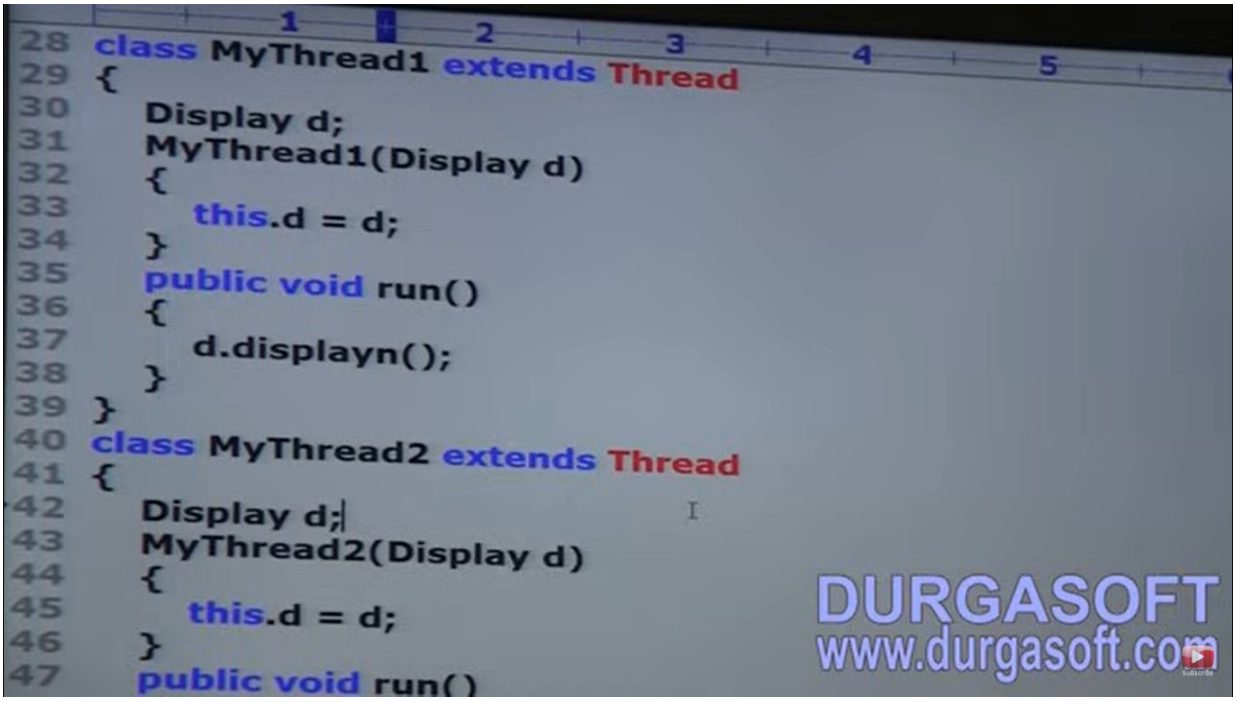
Example with synchronization 2: it is also a object level lock.



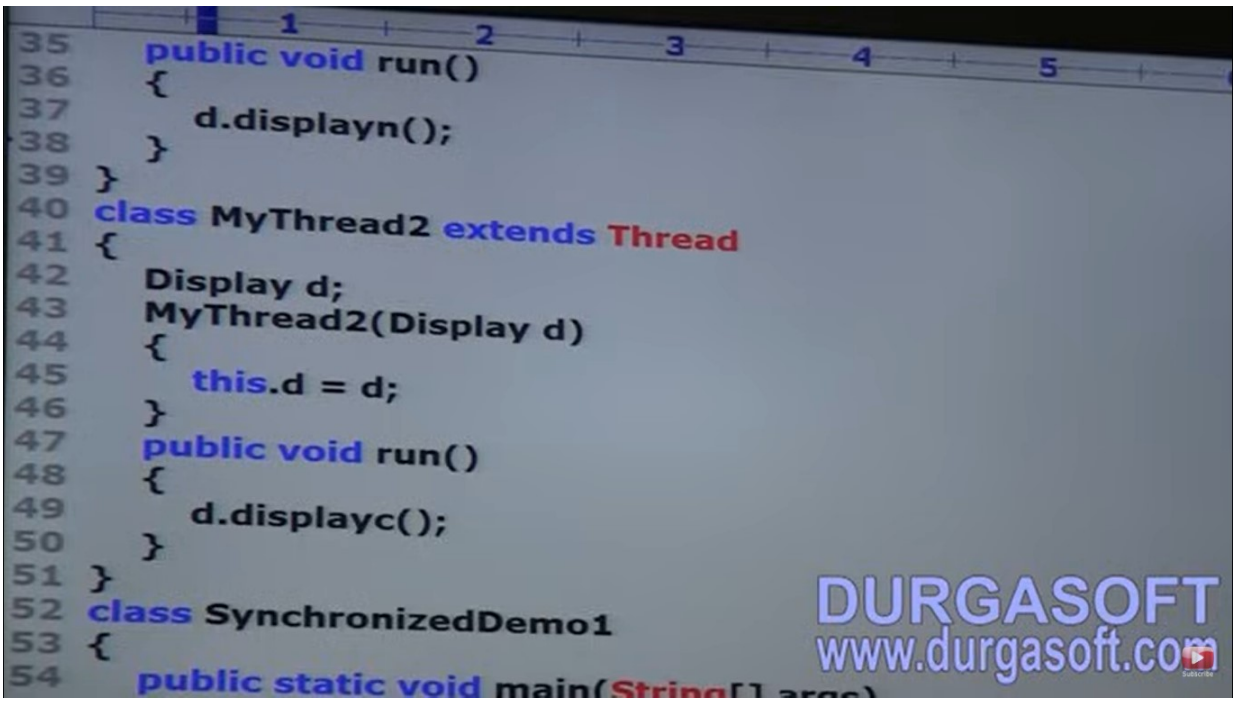
```
class Display
{
   public synchronized void displayn()
   {
      for(int i =1; i<=10; i++)
      {
         System.out.print(i);
         try
         {
               Thread.sleep(2000);
         }
         catch(InterruptedException e){}
      }
   }
   public synchronized void displayc()
   {
      for(int i =65; i<=75; i++)
      {
         System.out.print((char)i);
         try
```

```
   public synchronized void displayc()
   {
      for(int i =65; i<=75; i++)
      {
         System.out.print((char)i);
         try
         {
               Thread.sleep(2000);
         }
         catch(InterruptedException e){}
      }
   }
}
class MyThread1 extends Thread
{
   Display d;
   MyThread1(Display d)
   {
      this.d = d;
   }
```



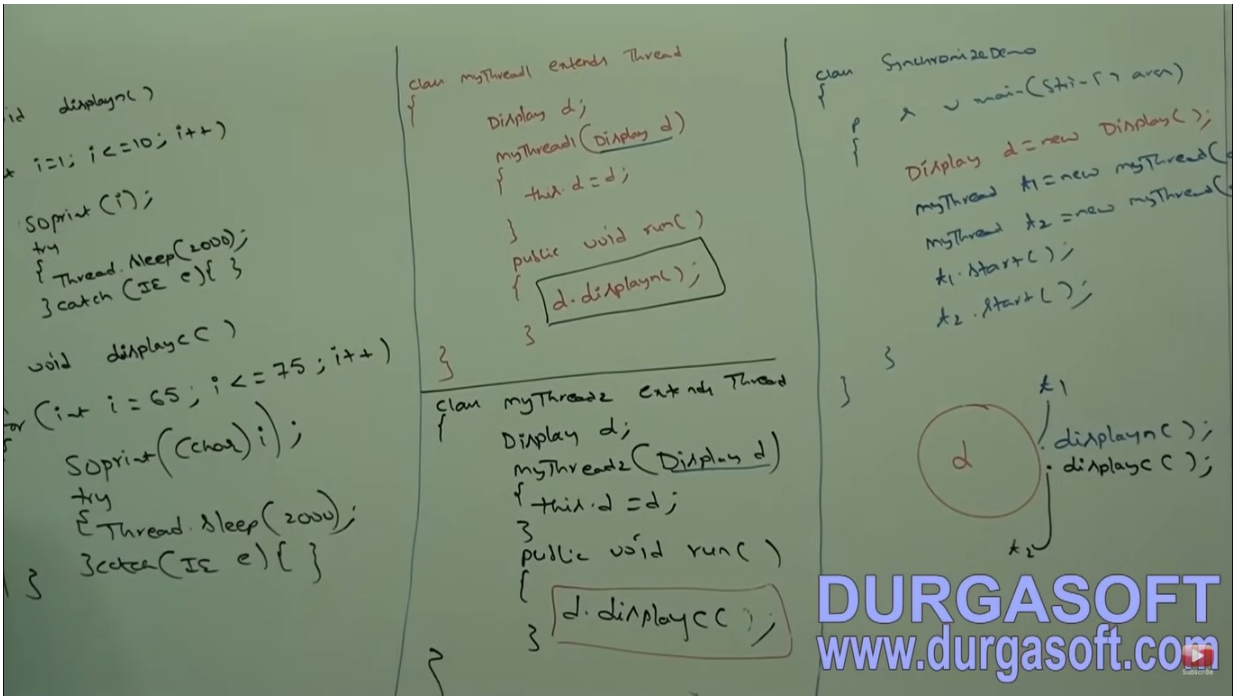
```
28 class MyThread1 extends Thread
29 {
30    Display d;
31    MyThread1(Display d)
32    {
33       this.d = d;
34    }
35    public void run()
36    {
37       d.displayn();
38    }
39 }
40 class MyThread2 extends Thread
41 {
42    Display d;
43    MyThread2(Display d)
44    {
45       this.d = d;
46    }
47    public void run()
```

```
35    public void run()
36    {
37       d.displayn();
38    }
39 }
40 class MyThread2 extends Thread
41 {
42    Display d;
43    MyThread2(Display d)
44    {
45       this.d = d;
46    }
47    public void run()
48    {
49       d.displayc();
50    }
51 }
52 class SynchronizedDemo1
53 {
54    public static void main(String[] args)
```

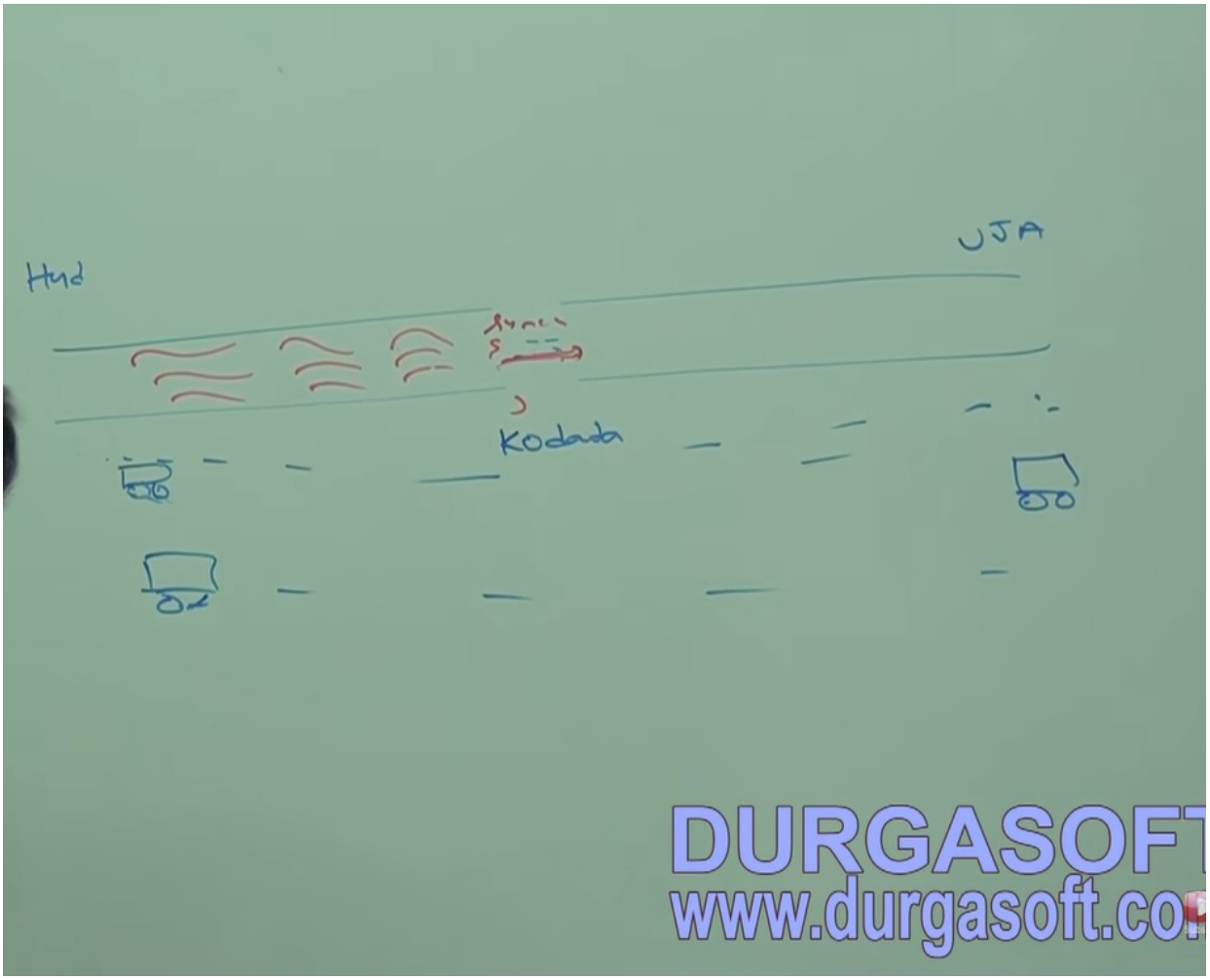
```
45          this.d = d;
46      }
47      public void run()
48      {
49          d.displayc();
50      }
51  }
52  class SynchronizedDemo1
53  {
54      public static void main(String[] args)
55      {
56          Display d = new Display();
57          MyThread1 t1 = new MyThread1(d);
58          MyThread2 t2 = new MyThread2(d);
59          t1.start();
60          t2.start();
61      }
62  }
63
```

DURGASOFT
www.durgasoft.com



SYNCHRONIZATION BLOCK

(3) STORY :



If very few lines of the codes required synchronization then it is not recommended to declare entire method as synchronizer we have enclose those few lines of the codes by using synchronizer block.

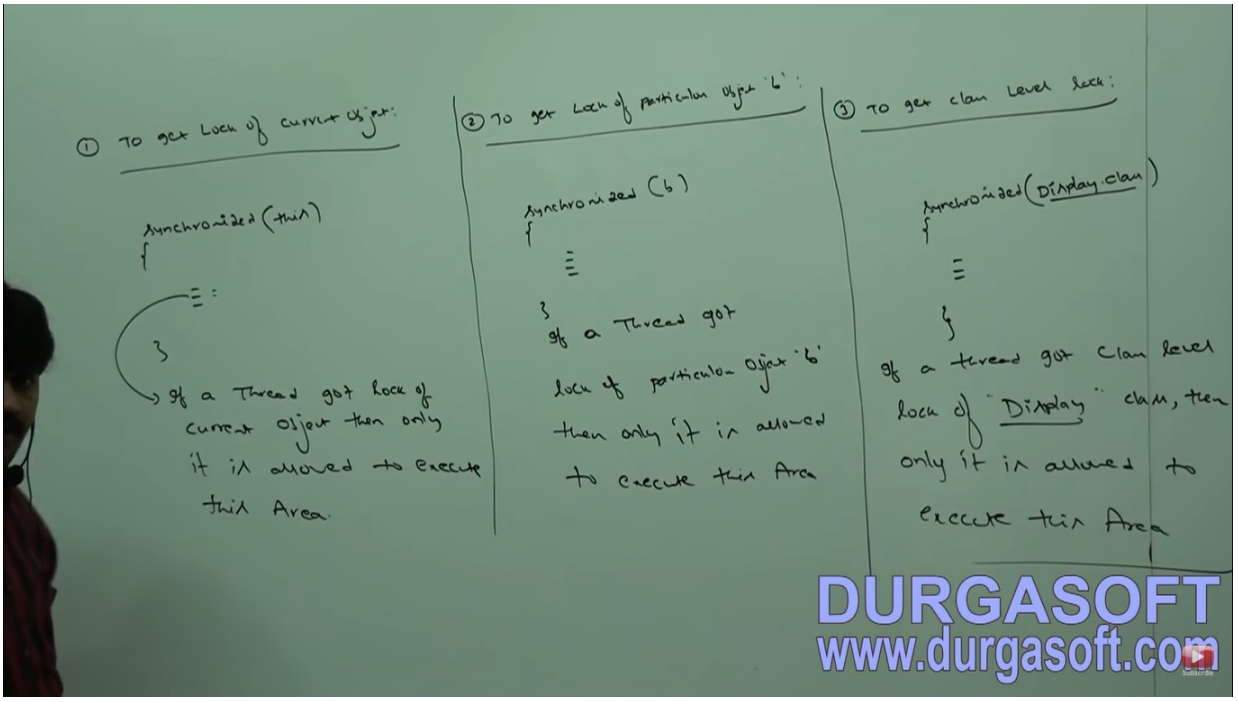
The main advantage of synchronizer block over synchronizer method is it reduces waiting time of thread and improves performance of the system.

We can declare synchronizer block as followed :

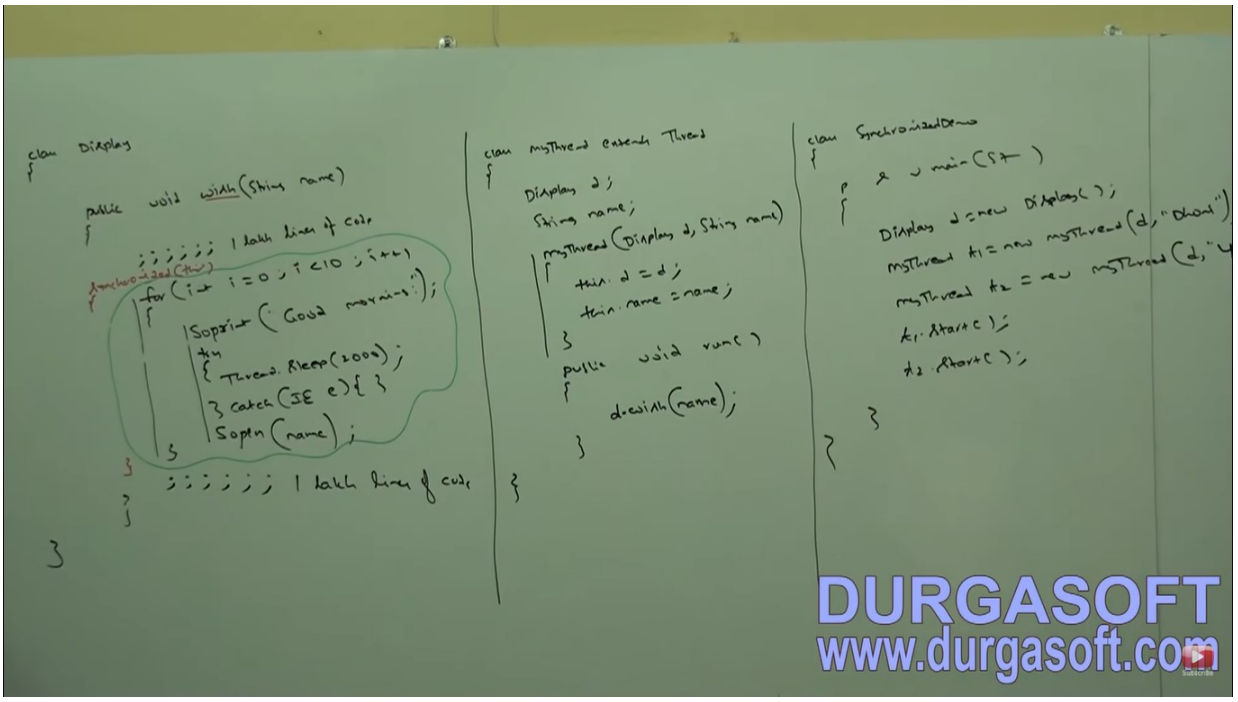
To get lock of current object :- (1)

To get lock of particular object :- (2)

To get lock of class level:- (3)

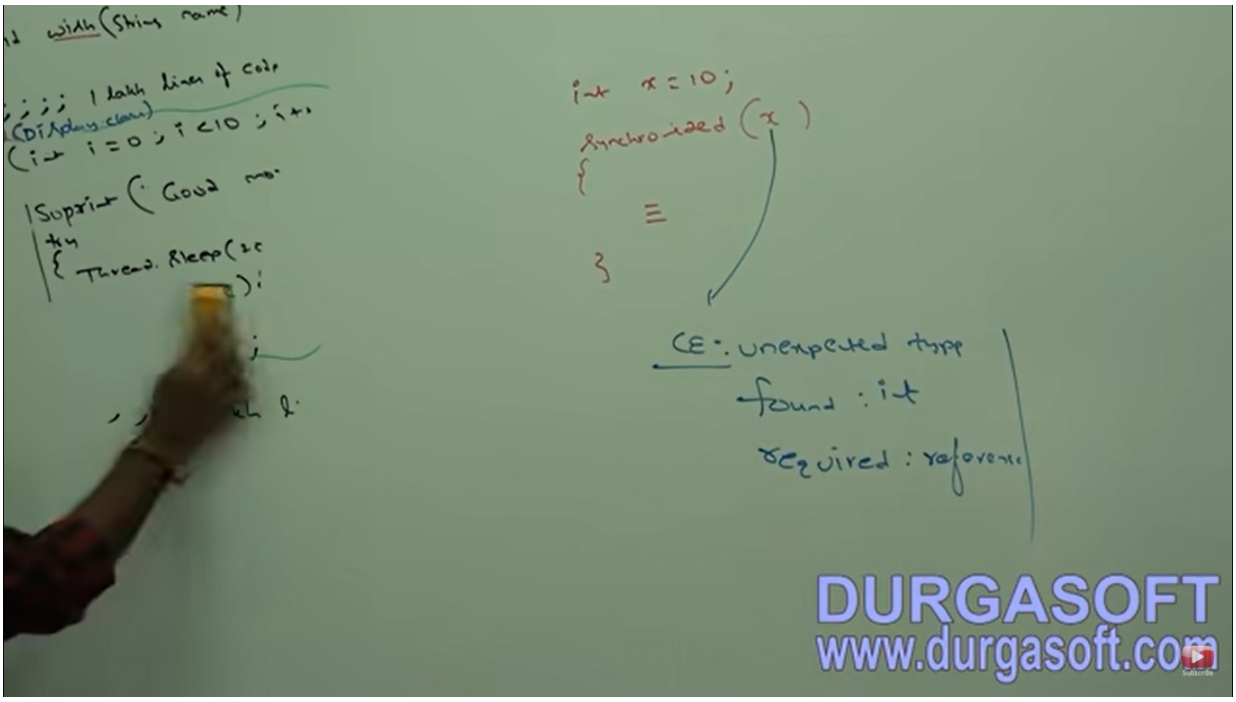


Ex – for to get lock of current object.



Lock concept applicable for object types and class types but not for primitives hence, we can't pass primitive type as argument to synchronization block otherwise we will get compile time error saying 'unexpected type found int , required reference '

Ex-



FAQ :

(1) what is synchronized keyword, where we can apply ?

(2) explain advantage of synchronized keyword?

(3) explain disadvantage of synchronized keyword?

(4) what is race condition?

If multiple threads are operating simultaneously on same java object then there may be a chance of data inconsistency problem this is called race codition.

We can overcome this problem by using synchronied keyword

(5) what is object lock and when it is required?

(6) what is class level lock and when it is required?

(7) what is the difference between class level lock and object level lock ?

(8) while a thread executing synchronized method on the given object , is remaining threads are allowed to execute any other synchronized method simultaneously on the same object?

No

(9) what is synchronized block ?

(10) how to declare synchronized block to get lock of current object?

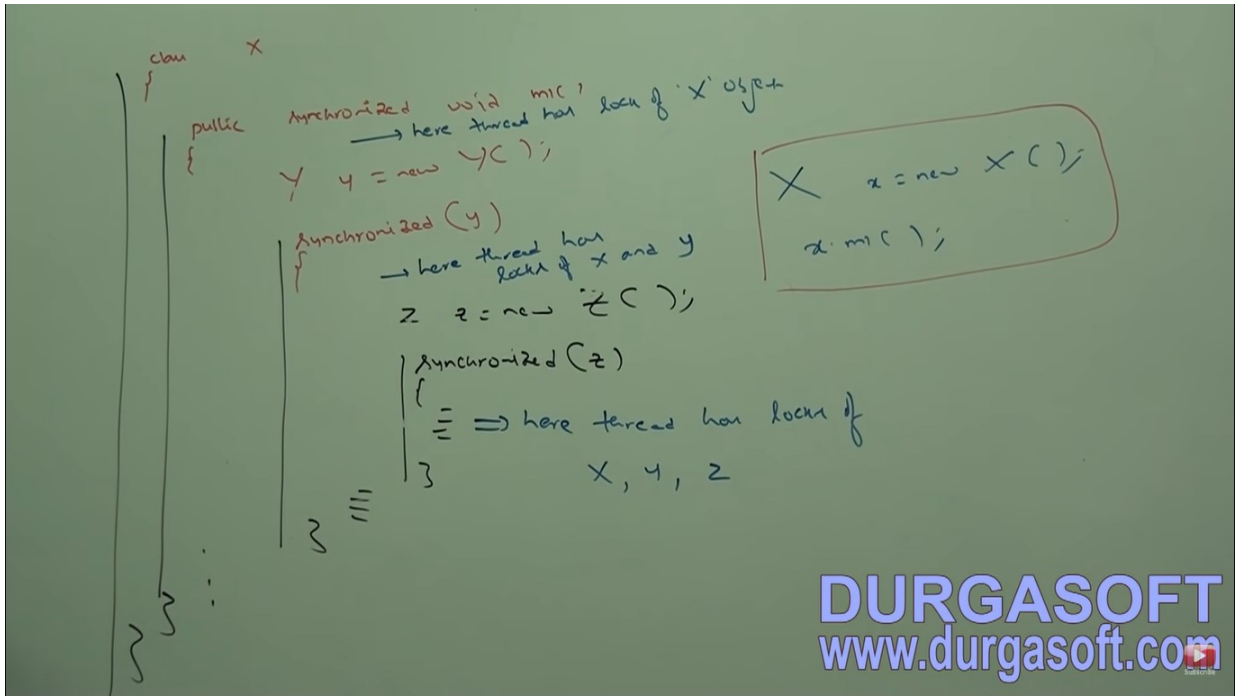
(11) how to declare synchronized block to get class level lock ?

(12) what is the advantage of synchronized block over synchronized method ?

(13) is a thread can aquired multiple locks simultaneously ?

Yes ofcourse from different objects.

Ex-



(14) what is synchronized statement (interview people created terminology?

The statements present in synchronized method and the synchronized block are called synchronized statements.